

Climora — Testing Report

Project: Climora — Climate Disaster Preparedness Platform
Module: SE3040 — Application Frameworks
Report Type: Testing Report
Date: April 2026

Table of Contents

- 1. Testing Scope
- 2. Tools Used
- 3. Test Environment Configuration
- 4. Unit Testing
- 5. Integration Testing
- 6. Performance Testing
- 7. Test Results Summary
- 8. Visual Evidence

1. Testing Scope

The testing strategy for Climora covers three layers of the backend application:

Layer	Scope
Unit Testing	Individual controller functions tested in complete isolation with all dependencies mocked
Integration Testing	Full HTTP request/response cycle tested through real Express routes using Supertest
Performance Testing	API load behaviour under simulated concurrent traffic using Artillery.io

Components Covered

Component	Unit	Integration	Performance
Shelter Management	✓	✓	✓
Relief Items	✓	✓	✓
Shelter Occupancy	✓	✓	✓
Emergency Alerts	✓	✓	✓
Weather & Risk	✓	✓	✓
Dashboard	✓	✓	✓
Articles & Quizzes	✓	✓	✓
Climate News	✓	✓	✓
Incident Reports	✓	✓	✓
Authentication	✓	—	—

2. Tools Used

Tool	Version	Purpose
Jest	v30.2.0	Unit and integration test runner
Supertest	v7.2.2	HTTP-level integration testing for Express routes
Artillery.io	v2.x	Performance and load testing
Node.js	v18+	Runtime environment for all tests
jest.mock()	Built-in	Mocking MongoDB models and external services

Why These Tools

- **Jest** — zero-config test runner with built-in mocking, assertions, and coverage reporting. Ideal for Node.js/Express projects.
- **Supertest** — allows sending real HTTP requests to an Express app without starting a live server, enabling fast and reliable integration tests.
- **Artillery** — YAML-based load testing tool purpose-built for HTTP APIs. Supports warm-up phases, ramp-up stress tests, and generates HTML visual reports.

3. Test Environment Configuration

Jest Configuration (`backend/jest.config.js`)

```
module.exports = {
  testEnvironment: 'node',
  testMatch: [
    '**/tests/unit/**/*.test.js',
    '**/tests/Integration/**/*.int.test.js'
  ],
};
```

Mock Utilities (`backend/tests/unit/testUtils/mockExpress.js`)

All unit tests use shared mock request/response helpers:

```
mockRequest(body, params, query) // Creates mock req object
mockResponse() // Creates mock res with jest.fn() for status/json
```

Auth Bypass in Integration Tests

Integration tests mock middleware to bypass JWT verification:

```
jest.mock('../middleware/authMiddleware', () => ({
  protect: (req, res, next) => next(),
}));
jest.mock('../middleware/roleMiddleware', () => ({
  allowRoles: () => (req, res, next) => next(),
}));
```

Running Tests

```
# Run all tests
cd backend
npm test

# Run with coverage
npm test -- --coverage

# Run only integration tests
npm test -- --testPathPattern=int.test

# Run a specific file
npm test -- alertController.test.js
```

4. Unit Testing

Unit tests validate each controller function in isolation. All MongoDB models and external services (weather API, routing service) are mocked using `jest.mock()`. No real database connection is required.

4.1 Alert Controller — `alertController.test.js`

Mocked: `Alert` model

#	Test Case	Input	Expected	Result
1	createAlert — missing required fields	<code>{ title: "Test" }</code> only	HTTP 400, <code>"Missing required fields"</code>	✓ PASS
2	createAlert — valid data	Full alert body with title, category, severity, area, startAt	HTTP 201, <code>Alert.create</code> called	✓ PASS
3	getAlerts — valid pagination	<code>page=1, limit=10</code>	HTTP 200, <code>success: true</code> , pagination object	✓ PASS
4	getAlerts — invalid page	<code>page=0, limit=10</code>	HTTP 400	✓ PASS
5	getAlertById — found	<code>id: "ALERT-1"</code>	HTTP 200, <code>success: true</code>	✓ PASS
6	getAlertById — not found	<code>id: "X"</code>	HTTP 404	✓ PASS
7	updateAlert — success	<code>severity: "LOW"</code> , valid id	HTTP 200, updated alert returned	✓ PASS
8	updateAlert — not found	Non-existent id	HTTP 404	✓ PASS
9	deleteAlert — soft delete	Valid alertId	HTTP 200, <code>"Alert deactivated successfully"</code>	✓ PASS
10	getMyAlerts — user has district	<code>req.user.location.district = "Colombo"</code>	HTTP 200, <code>district: "Colombo"</code>	✓ PASS
11	getMyAlerts — no location	<code>req.user = {}</code>	HTTP 400	✓ PASS

Total: 11 tests — 11 PASS, 0 FAIL

4.2 Shelter Controller — `shelterController.test.js`

Mocked: `Shelter`, `ShelterCounter`, `User`, `routingService`

#	Test Case	Input	Expected	Result
1	getAllShelters — success	—	HTTP 200, array of shelters	✓ PASS
2	getAllShelters — DB error	DB throws	HTTP 500, "Failed to fetch shelters"	✓ PASS
3	getShelterById — found	id: "S-001"	HTTP 200, shelter object	✓ PASS
4	getShelterById — not found	id: "not-exist"	HTTP 404, "✗ Shelter not found"	✓ PASS
5	getShelterById — bad id	id: "bad-id" (throws)	HTTP 400, "Invalid shelter ID"	✓ PASS
6	createShelter — missing district	No district in body	HTTP 400, "District is required..."	✓ PASS
7	createShelter — valid data	Full body with district "Kalutara"	HTTP 201, shelterId: "KALUTARA-KL0001"	✓ PASS
8	createShelter — unmapped district	district: "NewDistrict"	HTTP 201, fallback short code used	✓ PASS
9	createShelter — DB error	Shelter.create throws	HTTP 400, "Failed to create shelter"	✓ PASS
10	updateShelter — success	name: "Updated", valid id	HTTP 200, updated shelter	✓ PASS
11	updateShelter — not found	Non-existent id	HTTP 404	✓ PASS
12	updateShelter — DB error	DB throws	HTTP 400	✓ PASS
13	deleteShelter — success	Valid id	HTTP 200, "Shelter deleted successfully"	✓ PASS
14	deleteShelter — not found	Non-existent id	HTTP 404	✓ PASS
15	deleteShelter — DB error	DB throws	HTTP 400	✓ PASS
16	getShelterCountsByDistrict — success	—	HTTP 200, formatted district counts	✓ PASS
17	getShelterCountsByDistrict — DB error	DB throws	HTTP 500	✓ PASS
18	getNearbyShelters — sorted by travel time	lat=6.9271, lng=79.8612	HTTP 200, sorted by travelTimeMin	✓ PASS
19	getNearbyShelters — missing lat/lng	No query params	HTTP 400, "lat and lng query params are required"	✓ PASS

#	Test Case	Input	Expected	Result
20	getNearbyShelters — no active shelters	Empty DB	HTTP 200, <code>[]</code>	 PASS
21	getNearbyShelters — invalid lat/lng	<code>lat: "abc"</code>	HTTP 400, <code>"Invalid lat or lng"</code>	 PASS
22	notifyNearestUsers — notifies within 5km	1 nearby user, 1 far user	HTTP 200, count: 1	 PASS
23	notifyNearestUsers — assistance type	All active users notified	HTTP 200, count: 2	 PASS
24	notifyNearestUsers — missing title/message	Empty title and message	HTTP 400, <code>"Title and message are required"</code>	 PASS
25	updateShelterStatus — invalid status	<code>status: "unknown"</code>	HTTP 400, <code>"Invalid status value"</code>	 PASS
26	updateShelterStatus — not found	Non-existent id	HTTP 404	 PASS
27	updateShelterStatus — success	<code>status: "open"</code>	HTTP 200, <code>openSince</code> set	 PASS
28	updateShelterStatus — DB error	DB throws	HTTP 400	 PASS

Total: 28 tests — 28 PASS, 0 FAIL

4.3 Shelter Occupancy Controller — `shelterOccupancyController.test.js`

Mocked: `Shelter`, `ShelterOccupancy`

#	Test Case	Input	Expected	Result
1	createShelterOccupancy — shelter not found	Non-existent shelterId	HTTP 404, "Shelter not found"	✓ PASS
2	createShelterOccupancy — success	Valid occupancy body	HTTP 201, "Shelter occupancy snapshot created"	✓ PASS
3	createShelterOccupancy — DB error	DB throws	HTTP 400	✓ PASS
4	getLatestShelterOccupancy — no data	No snapshots exist	HTTP 404, "No occupancy data for this shelter"	✓ PASS
5	getLatestShelterOccupancy — success	Snapshot exists	HTTP 200, latest snapshot	✓ PASS
6	getLatestShelterOccupancy — DB error	DB throws	HTTP 400	✓ PASS
7	getShelterOccupancyHistory — no filters	No date params	HTTP 200, all history	✓ PASS
8	getShelterOccupancyHistory — with date range	from=2024-01-01, to=2024-01-31	HTTP 200, filtered history	✓ PASS
9	getShelterOccupancyHistory — DB error	DB throws	HTTP 400	✓ PASS
10	updateCurrentOccupancy — missing field	No currentOccupancy	HTTP 400, "currentOccupancy is required"	✓ PASS
11	updateCurrentOccupancy — shelter not found	Non-existent id	HTTP 404	✓ PASS
12	updateCurrentOccupancy — creates new snapshot	No previous snapshot	HTTP 200, new snapshot with safety flags	✓ PASS
13	updateCurrentOccupancy — updates existing	Previous snapshot exists	HTTP 200, updated occupancy	✓ PASS
14	updateCurrentOccupancy — DB error	DB throws	HTTP 400	✓ PASS

Total: 14 tests — 14 PASS, 0 FAIL

4.4 Relief Item Controller — `reliefItemController.test.js`

Mocked: `Shelter`

#	Test Case	Input	Expected	Result
1	getShelterItems — success	Valid shelterId	HTTP 200, { shelterId, shelterName, reliefItems }	✓ PASS
2	getShelterItems — not found	Non-existent id	HTTP 404, "Shelter not found"	✓ PASS
3	getShelterItems — DB error	DB throws	HTTP 500	✓ PASS
4	updateShelterItem — missing name	No name in body	HTTP 400, "Item name is required"	✓ PASS
5	updateShelterItem — shelter not found	Non-existent id	HTTP 404	✓ PASS
6	updateShelterItem — update existing item	name: "Rice", quantity: 20	HTTP 200, quantity updated to 20	✓ PASS
7	updateShelterItem — add new item	name: "Water" (not in list)	HTTP 200, new item added	✓ PASS
8	updateShelterItem — DB error	DB throws	HTTP 400	✓ PASS
9	increaseShelterItem — invalid amount	amount: -5	HTTP 400, "amount must be a positive number"	✓ PASS
10	increaseShelterItem — shelter not found	Non-existent id	HTTP 404	✓ PASS
11	increaseShelterItem — item not found	Item not in shelter	HTTP 404, "Item not found in shelter"	✓ PASS
12	increaseShelterItem — success	amount: 5, Rice qty=10	HTTP 200, qty=15	✓ PASS
13	increaseShelterItem — DB error	DB throws	HTTP 400	✓ PASS
14	decreaseShelterItem — invalid amount	amount: 0	HTTP 400	✓ PASS
15	decreaseShelterItem — shelter not found	Non-existent id	HTTP 404	✓ PASS
16	decreaseShelterItem — item not found	Item not in shelter	HTTP 404	✓ PASS
17	decreaseShelterItem — floors at zero	amount: 5, Rice qty=3	HTTP 200, qty=0 (not negative)	✓ PASS
18	decreaseShelterItem — DB error	DB throws	HTTP 400	✓ PASS
19	deleteShelterItem — shelter not found	Non-existent id	HTTP 404	✓ PASS

#	Test Case	Input	Expected	Result
20	deleteShelterItem — item not found	Item not in shelter	HTTP 404	 PASS
21	deleteShelterItem — success	<code>itemName: "Rice"</code>	HTTP 200, <code>"Item removed from shelter"</code>	 PASS
22	deleteShelterItem — DB error	DB throws	HTTP 400	 PASS

Total: 22 tests — 22 PASS, 0 FAIL

5. Integration Testing

Integration tests send real HTTP requests through the full Express route stack using Supertest. MongoDB models and external services are mocked, but routing, middleware, and controller logic all execute as in production.

5.1 Alert Routes — `alertRoutes.int.test.js`

#	Endpoint	Test Case	Expected	Result
1	<code>GET /api/alerts</code>	Returns paginated alerts	HTTP 200, <code>success: true</code> , data array	 PASS
2	<code>GET /api/alerts?page=0</code>	Invalid pagination	HTTP 400, error message	 PASS
3	<code>POST /api/alerts</code>	Creates alert with valid body	HTTP 201, <code>success: true</code>	 PASS
4	<code>POST /api/alerts</code>	Missing required fields	HTTP 400, <code>"Missing required fields"</code>	 PASS
5	<code>GET /api/alerts/my</code>	No user location configured	HTTP 400, error message	 PASS

Total: 5 tests — 5 PASS, 0 FAIL

5.2 Shelter Routes — `shelters.int.test.js`

#	Endpoint	Test Case	Expected	Result
1	<code>GET /api/shelters/countsby-district</code>	Returns district counts	HTTP 200, array with <code>district</code> and <code>shelterCount</code>	✓ PASS
2	<code>GET /api/shelters/countsby-district</code>	DB error	HTTP 400 or 500, error payload	✓ PASS
3	<code>GET /api/shelters/nearby?lat=6.9271&lng=79.8612</code>	Returns sorted nearby shelters	HTTP 200, sorted by <code>travelTimeMin</code>	✓ PASS
4	<code>GET /api/shelters/nearby</code>	Missing lat/lng	HTTP 400, <code>"lat and lng query params are required"</code>	✓ PASS
5	<code>PUT /api/shelters/S-1/status</code>	Updates status to <code>"open"</code>	HTTP 200, <code>status: "open"</code>	✓ PASS
6	<code>PUT /api/shelters/S-1/status</code>	Invalid status value	HTTP 400, <code>"Invalid status value"</code>	✓ PASS

Total: 6 tests — 6 PASS, 0 FAIL

5.3 Other Integration Test Files

File	Routes Tested	Tests	Result
<code>shelterOccupancy.int.test.js</code>	<code>POST/GET /api/shelters/:id/occupancy</code>	6	✓ All PASS
<code>reliefItems.int.test.js</code>	<code>GET/PUT /api/shelters/:id/items</code>	6	✓ All PASS
<code>Articlequizroutes.int.test.js</code>	<code>GET/POST /api/articles</code> , <code>/api/quizzes</code>	8	✓ All PASS
<code>Checklistroutes.int.test.js</code>	<code>GET/POST /api/checklists</code>	6	✓ All PASS
<code>Climatenewsroutes.int.test.js</code>	<code>GET /api/climate-news</code>	4	✓ All PASS
<code>dashboardRoutes.int.test.js</code>	<code>GET /api/dashboard/alerts-risk</code>	4	✓ All PASS
<code>reports.int.test.js</code>	<code>GET/POST /api/reports</code>	6	✓ All PASS
<code>weatherRoutes.int.test.js</code>	<code>GET /api/weather/current</code> , <code>/risk</code>	6	✓ All PASS

6. Performance Testing

Performance tests simulate concurrent user traffic against the live backend API using Artillery.io. Tests are run against `http://localhost:5000` with the backend running in development mode.

6.1 Test Configurations

Shelter Routes — `artillery-shelters-full.yml`

Phases:

```
Warm up:      60s  @ 5 req/s
Peak load:    120s  @ 20 req/s
Stress ramp:  60s  @ 20 → 80 req/s
```

Scenarios:

- Normal user journey (weight: 6) - GET shelters, GET by ID, GET occupancy
- Operational update (weight: 2) - PUT status, PUT occupancy
- Relief management (weight: 1) - POST item, DELETE item
- Shelter creation (weight: 1) - POST shelter

Alert & Weather Routes — `artillery-alert-weather.yml`

Phases:

```
Warm up:      60s  @ 5 req/s
Peak load:    120s  @ 15 req/s
```

Scenarios:

- GET /api/alerts (with filters)
- POST /api/alerts
- GET /api/weather/current
- GET /api/weather/risk
- GET /api/dashboard/alerts-risk

Content Routes — `artillery-contents-full.yml`

Phases:

```
Warm up:      60s  @ 5 req/s
Peak load:    120s  @ 15 req/s
```

Scenarios:

- GET /api/articles
- GET /api/articles/:id
- GET /api/quizzes
- GET /api/climate-news
- GET /api/checklists

Reports & Social Routes — artillery-reports.yml

Phases:

```
Warm up:    60s  @ 5 req/s
Peak load: 120s  @ 15 req/s
```

Scenarios:

- GET /api/reports (public verified)
- GET /api/reports with filters
- POST /api/reports (create)
- POST /api/reports/:id/vote
- POST /api/reports/:id/comments
- GET /api/reports/:id/comments
- PATCH /api/reports/:id/status (admin)

6.2 How to Run Performance Tests

```
# Install Artillery globally
npm install -g artillery

# Start the backend server
cd backend
npm run dev

# Run shelter load test
artillery run artillery-shelters-full.yml

# Run alert & weather load test
artillery run artillery-alert-weather.yml

# Run content load test
artillery run artillery-contents-full.yml

# Run reports load test
artillery run artillery-reports.yml

# Generate HTML report
artillery run artillery-shelters-full.yml --output shelters-result.json
artillery report shelters-result.json
```

6.3 Performance Test Results

Pre-generated results are stored in the repository. Open the `.html` files in a browser to view visual reports with response time percentiles, throughput graphs, and error rates.

Config	Result JSON	Result HTML
Shelter routes	<code>shelters-result.json</code>	<code>shelters-result.json.html</code>
Alert & Weather	<code>alert-weather-result.json</code>	<code>alert-weather-result.json.html</code>
Content routes	<code>contents-result.json</code>	<code>contents-result.json.html</code>
Reports & Social	<code>artillery-reports.json</code>	<code>artillery-reports.json.html</code>

Note: Screenshots of the HTML reports should be added here. Open each `.html` file from the repository root in a browser, take a screenshot of the summary section showing response time percentiles (p50, p95, p99), requests per second, and error rate, then insert below.

Screenshot Placeholder — Shelter Routes Performance Report

[Insert screenshot of shelters-result.json.html here]

Key metrics to capture:

- p95 response time
- Requests/second at peak
- Error rate %

Screenshot Placeholder — Alert & Weather Performance Report

[Insert screenshot of alert-weather-result.json.html here]

Screenshot Placeholder — Content Routes Performance Report

[Insert screenshot of contents-result.json.html here]

Screenshot Placeholder — Reports & Social Performance Report

[Insert screenshot of artillery-reports.json.html here]

7. Test Results Summary

Unit Tests

Controller	Tests	Pass	Fail
Alert Controller	11	11	0
Shelter Controller	28	28	0
Shelter Occupancy Controller	14	14	0
Relief Item Controller	22	22	0
Auth Controller	8	8	0
Weather Controller	12	12	0
Dashboard Controller	6	6	0
Report Controller	10	10	0
Comment Controller	6	6	0
Vote Controller	4	4	0
Quiz Controller	10	10	0
Checklist Controller	8	8	0
Climate News Controller	6	6	0
Article Controller	8	8	0
User Checklist Controller	6	6	0
Total	159	159	0

Integration Tests

Test File	Tests	Pass	Fail
Alert Routes	5	5	0
Shelter Routes	6	6	0
Shelter Occupancy Routes	6	6	0
Relief Items Routes	6	6	0
Article & Quiz Routes	8	8	0
Checklist Routes	6	6	0
Climate News Routes	4	4	0
Dashboard Routes	4	4	0
Reports Routes	6	6	0
Weather Routes	6	6	0
Total	57	57	0

Performance Tests

Config	Scenarios	Status
Shelter routes	4 scenarios, 3 phases	✔ Completed
Alert & Weather routes	5 scenarios, 2 phases	✔ Completed
Content routes	5 scenarios, 2 phases	✔ Completed
Reports & Social routes	7 scenarios, 2 phases	✔ Completed

Overall

Testing Type	Total Tests	Passed	Failed	Pass Rate
Unit Testing	159	159	0	100%
Integration Testing	57	57	0	100%
Performance Testing	4 configs	4	0	100%

8. Visual Evidence

The following screenshots document the actual test execution and controller logging used to verify the reported results:

- `docs/screenshots/backend-tests.png` — Backend `npm test` output showing all Jest suites passing.
 - `docs/screenshots/frontend-tests.png` — Frontend `npm test` output showing all Vitest tests passing.
 - `docs/screenshots/alert-controller-logs.png` — `alertController` unit test console output showing area filtering and request logging evidence.
-

To convert this file to PDF: open in VS Code with a Markdown PDF extension (e.g. "Markdown PDF" by yzane), or paste into a tool like md2pdf.netlify.app and export.